

Switch-case is used to execute **different logic based on a single value** in a clean and organized way. In this project, it checks the input `<field's id` and applies the correct validation rule for that specific `<field`. Instead of writing many `if-else` statements, switch-case makes the code easier to read, manage, and extend. It also improves performance and reduces logical errors by clearly separating validation rules for each input `<field`.

When a user types or leaves an input `<field`, the `validateField()` function is triggered, and `switch(field.id)` identifies which `<field` is being checked, such as name, age, or phone number. Each case contains validation logic specific to that `<field`, and the `break` statement ensures that only the matched case runs. Additionally, **live validation** is implemented using `input` and `blur` events, which validate the user's input in real time and immediately display or remove error messages, improving user experience and data accuracy.

Edit Student Function

Explanation (Presentation style)

The `editStudent()` function allows the user to update an existing student's details. When the edit button is clicked, the function finds the selected student from the `students` array using the student ID. Once found, the form fields are automatically filled with the student's existing data. The `editId` variable is set to track that the form is now in edit mode, and the cancel button is displayed so the user can exit editing if needed.

View Student Function (Modal View)

Explanation

The `viewStudent()` function is used to display a student's details in a modal window. When triggered, it locates the selected student and inserts their data into the modal fields. The modal is then displayed using CSS flex layout, allowing users to view details without leaving the page.

Modal Close Functionality

Explanation

The modal can be closed in two ways: by clicking the close button or by clicking outside the modal content area. This is handled using event listeners that hide the modal when these actions occur, ensuring smooth and intuitive interaction.

Search Functionality

Explanation

The search feature allows users to find students by name in real time. As the user types in the search input field, the student list is filtered based on matching names and the table is re-rendered instantly.

Filter by Level

Explanation

The filter feature allows users to view students based on their academic level. When a level is selected, only students belonging to that level are displayed. If no level is selected, all students are shown again.

Sort by Age

Explanation

The `sortByAge()` function sorts the student list in ascending order based on age. A copy of the students array is used to avoid modifying the original data. The sorted list is then displayed in the table.

